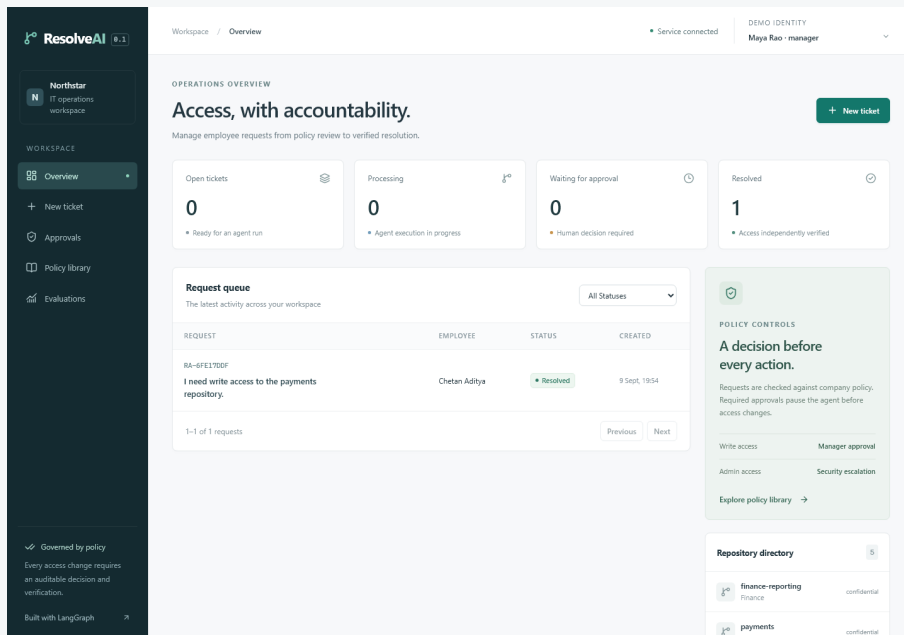


ResolveAI

Agentic AI Reliability & IT Access Automation

A production-style system for policy-grounded agent workflows, human approvals, verified actions, and automated evaluation.

The model proposes. Deterministic controls authorize. Observed state determines success.



A request becomes an evidence-backed workflow

An employee asks for repository access. ResolveAI grounds the identity and resource, retrieves policy, pauses for approval when required, executes a simulated change, reads permission independently, and evaluates the recorded outcome.

Designed and developed by Chetan Aditya. Bounded local demonstration; no paid live-model validation or production repository integration is claimed.

A plausible answer is not proof.

Reliability depends on what happened beyond the model response.

Ground the request

An employee name and repository mentioned in free text may be ambiguous or wrong. The workflow resolves both against stored catalog records and checks existing access.

Authorize the action

Retrieved text can explain policy, but it cannot grant authority. Reviewed evidence, deterministic policy checks and the current actor determine the allowed next step.

Prove the effect

A successful tool response alone does not establish that access changed. ResolveAI reads permission again through a separate tool before closing the ticket.

Make failure inspectable

Unknown resources, rejected approvals, stale evidence and mismatches route to recorded failure or escalation. The system preserves evidence for review and evaluation.

The v0.1 domain is intentionally narrow: repository access in a simulated enterprise environment. It is an engineering study of reliable action, not general IT troubleshooting.

One request. Explicit checkpoints.

Example: "I need write access to the payments repository."

01

Identify

Ground requester, classify intent and resolve payments in the repository catalog.

02

Inspect

Read current permission and retrieve relevant policy evidence.

03

Decide

Determine that write access requires an eligible manager approval.

04

Pause

Persist the approval and native LangGraph interrupt; permission remains unchanged.

05

Resume

After a recorded decision, resume the same run and revalidate action scope.

06

Verify

Execute the simulated grant, read permission independently, then resolve or escalate.

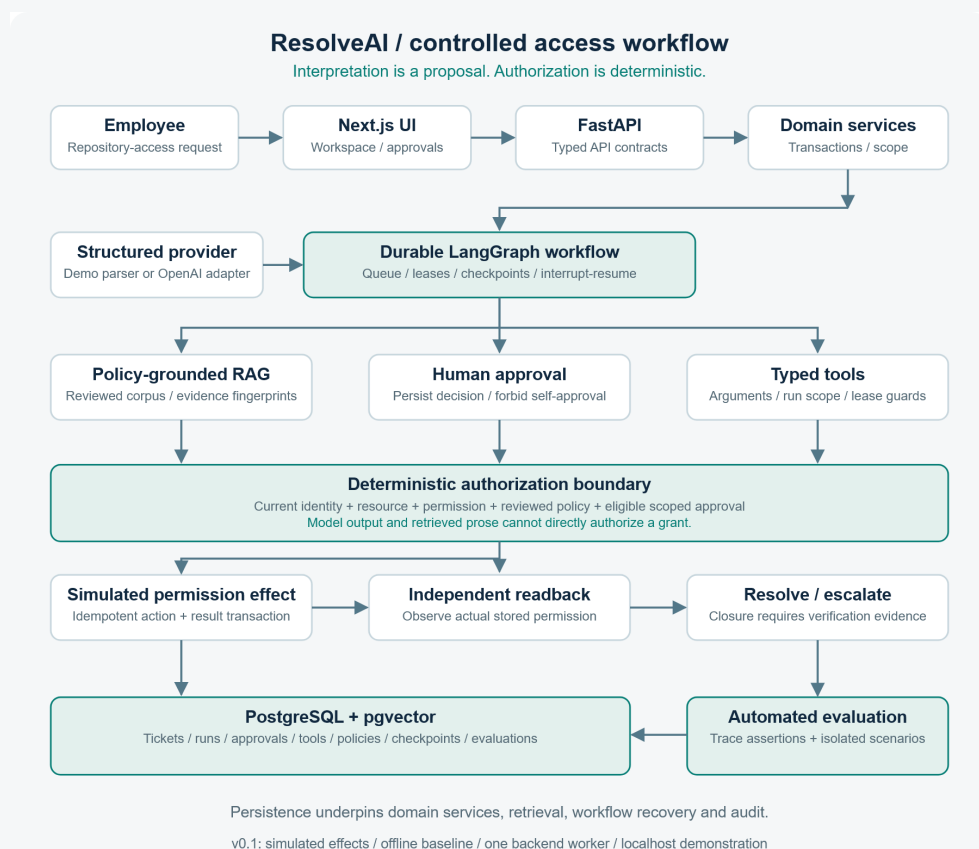
07

Evaluate

Check persisted identity, policy, approval, tool and terminal-state evidence.

Separate interpretation from authority.

A small full-stack system with explicit service and persistence boundaries.



One application boundary, durable state

Next.js communicates with FastAPI. Domain services own transactions; a single backend worker executes the LangGraph workflow. PostgreSQL stores domain records and checkpoints; pgvector ranks policy chunks.

Why the separation matters

Request, run, approval, tool result, permission, verification and evaluation remain independently inspectable. A generated answer cannot bypass mutation guards or manufacture closure evidence.

Evidence informs. Controls decide.

RAG supports the decision without becoming the authorization mechanism.

Policy-grounded retrieval

Eight Markdown policy sources become 35 section chunks. The seed process indexes the corpus; source changes invalidate stale chunks. Retrieval records its configuration, ranks vectors with pgvector and saves the selected evidence.

Reviewed policy fingerprints

Deterministic checks bind applicable policy to reviewed document and section fingerprints. Missing, changed or inapplicable evidence cannot silently relax a permission rule. Mutations recheck policy and current catalog facts.

Structured provider contracts

The provider classifies requests and proposes typed repository-tool arguments. Scope validation rejects changed resources, unknown tools or extra arguments. Requester identity comes from the stored ticket.

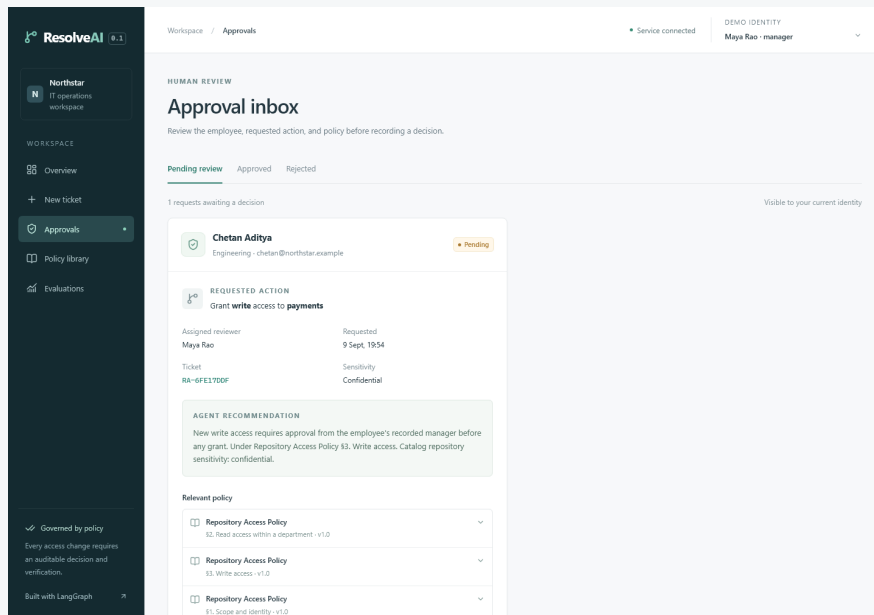
Offline by design: the shipped demo provider is a narrow deterministic parser. `local_hash` produces lexical vectors, not semantic embeddings. The OpenAI adapter exists; live paid behavior remains unverified.

Agent boundaries

The model does not authorize access, discover arbitrary actions, execute SQL, or directly control the permission service. Admin requests escalate rather than receiving an automatic grant.

Waiting is persistent state.

A human decision survives process restarts and resumes the original run.



A scoped, durable decision

Approval records bind employee, repository, permission and policy snapshot to an eligible reviewer. Self-approval is blocked. The current manager or an active administrator is checked against stored identity, not model text.

Native interrupt and resume

Approval creation is a completed node before the interrupt. The worker publishes the waiting state after checkpointing. Recording the decision and queuing the existing run share a database transaction; resume rereads that decision.

Conflicts and rejection

Same-decision retries preserve the original audit record; conflicting decisions are rejected. A rejected request escalates without a grant. A changed scope requires a new approval.

A tool result is only one piece of evidence.

Idempotency, recovery and independent verification support truthful closure.

Typed tools with execution guards

Validated arguments are checked against the owning run. Mutating tools require the designated workflow node and active lease. Identity, access, reviewed policy and approval eligibility are rechecked immediately before the effect.

Replay-safe simulated effects

An idempotency key identifies the action and rejects reuse with different arguments. The simulated permission effect and tool-result record share a transaction. Recovery reuses committed results if a later checkpoint write fails.

Read after write

A separate permission read records observed access. Closure requires the successful verification tool, its completed graph step and matching current permission. A state flag or optimistic success message cannot close the ticket.

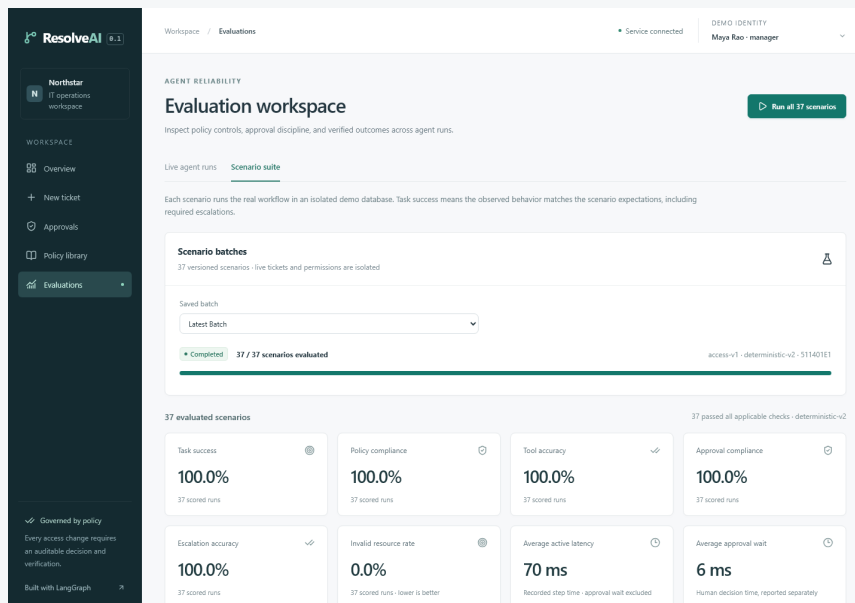
External-system boundary: this transactional guarantee applies to simulated effects in the local database. A real GitHub adapter needs reconciliation for uncertain network outcomes and external writes.

Recovery without additional infrastructure

Persisted pending runs, lease ownership, saved step deltas and checkpoints support bounded recovery. One backend worker process is supported; no broker or distributed service topology was added for v0.1.

Evaluate the trace, not the confidence.

Independent assertions compare recorded behavior with explicit expectations.



Two complementary views

Live-run evaluation checks evidence after a terminal run. Scenario batches execute the real graph in isolated temporary databases with seeded fixtures and injected failures, saving expected properties and complete traces.

What the evaluator checks

Correct identity and resource grounding; reviewed policy evidence; expected and forbidden tool use; approval before grant; verification before close; safe escalation; and correct terminal state.

Interpret the score carefully

A safely rejected request can pass its assertions without achieving access. The fixed 37-scenario suite is deterministic and bounded. Its score does not establish broad LLM reliability, semantic retrieval quality or enterprise safety.

Measured scope. Clear limits.

Current local validation - 9 September 2026; offline providers only.

Backend tests

490 passed, including PostgreSQL / pgvector integration

Chromium E2E

12 passed against the production frontend build

Evaluation scenarios

37 / 37 passed with demo + local_hash providers

Recovery & deployment

HTTP restart recovery passed; Compose config passed. Docker execution blocked by unavailable local engine.

Implemented technology

AI / workflow: LangGraph, structured outputs, RAG, typed tools, approvals, evaluation.

Backend: Python 3.12, FastAPI, Pydantic, SQLAlchemy, Alembic.

Data: PostgreSQL 16 and pgvector; SQLite for isolated tests.

Frontend: Next.js, React, TypeScript, Tailwind CSS.

Engineering: Docker Compose, pytest, Ruff, Playwright, GitHub Actions definitions.

Exact commands, scope and limitations: [portfolio evaluation section](#) and docs/release-validation.md in the source distribution. Local results do not imply a completed remote CI run.

A credible v0.1 has boundaries.

Designed and developed by Chetan Aditya.

Current limitations

Simulated repository effects; no production GitHub/Jira integration or enterprise SSO; incomplete public route authorization; no multi-tenant platform or general IT troubleshooting. Evidence is persisted but not a tamper-proof ledger. Live paid-model execution remains a separate validation gate.

Roadmap tied to evidence

v0.2: controlled live-model benchmark, broader natural language fixtures, semantic embedding comparison, and measured latency/cost.

v0.3: GitHub sandbox adapter, external-effect reconciliation, approval expiry and stronger identity boundaries.

Later: additional domains, multi-worker execution, streaming and production authentication when justified.

About the developer

Chetan Aditya

AI Engineer working across agentic AI, AI evaluation, AI reliability, multimodal AI, LLM systems, and practical AI engineering.

[Portfolio](#) / [GitHub](#) / [LinkedIn](#) / [Resume](#)

Explore the project: read the architecture and evaluation guides, run the localhost demo, and inspect the stored approval and verification evidence. The case study includes a video section; the recording is coming soon.

Case study: chetanaditya.netlify.app/projects/resolveai/

Contact: chetan.lakka@gmail.com